

SOFTWARE ENGINEERING PROJECT

**KUru: Intelligent PLO-to-Career Navigator for Kasetsart University
using RAG and Knowledge Graph Reasoning**

BY

**Thanawat [Surname]
Phantawat Luengsiriwattana**

**DEPARTMENT OF COMPUTER ENGINEERING
FACULTY OF ENGINEERING
KASETSART UNIVERSITY
Academic Year 2568**

Abstract

KUru is an AI-powered academic pathway navigation system for Kasetsart University (KU). The system addresses a clearly defined problem: students who are interested in exploring career paths or programs at KU lack personalized, structured guidance when selecting programs for TCAS applications.

The system employs a Retrieval-Augmented Generation (RAG) pipeline grounded in official KU curriculum documents (มคอ.2) to enable semantic question-answering over program content in both Thai and English. A Neo4j knowledge graph maps KU faculties, Program Learning Outcomes (PLOs), skill clusters, and career paths, enabling multi-hop reasoning from a student's interest profile to personalized program recommendations. An adaptive interest discovery interface combining an attribute-based topic grid and a single structured follow-up question builds the student profile. The system serves prospective students only (pre-admission); features for enrolled KU students are out of scope.

The system covers all KU programs and is evaluated using the RAGAS framework for RAG quality and Mean Reciprocal Rank (MRR) and NDCG@5 for recommendation quality.

Chapter 1: Introduction

1.1 Background

Thailand's university admission system, TCAS (Thai University Central Admission System), requires students to select and apply to specific faculties at specific universities through up to four rounds, each with distinct requirements, quotas, and criteria. Kasetsart University (KU), one of Thailand's leading public universities, offers programs spanning agriculture, engineering, science, humanities, business, and more across its campuses.

Despite the availability of official information about each program, most prospective students make faculty selection decisions with limited structured guidance. Program Learning Outcomes (PLOs), which formally document the competencies a graduate will have developed, are defined for every accredited program in Thailand via the TQF (Thai Qualifications Framework) document system, known as มคอ.2. These documents are the authoritative source of what a program actually teaches and produces, yet they are inaccessible to the students they most directly affect: written in formal Thai, formatted inconsistently across faculties, and distributed as PDFs on faculty websites without any structured interface.

Prospective students who are interested in KU programs — whether finishing high school, taking a gap year, or reconsidering their options — face this problem acutely, as they must make high-stakes decisions with limited structured guidance.

1.2 Problem Statement

This project addresses two related problems:

- High school students selecting KU programs for TCAS lack personalized, grounded guidance that connects their interests to specific program PLOs, career outcomes, and admission requirements. Existing tools are either generic (general-purpose LLMs with no KU-specific data) or static (official faculty web pages with no personalization or synthesis).
-

The consequence of the first problem is suboptimal faculty selection, contributing to higher rates of program dissatisfaction, transfer requests, and dropout. The consequence of the second is that graduates are unable to clearly articulate the competencies their degree has produced, weakening their employability.

1.3 Solution Overview

KUru is a bilingual (Thai and English) web application that serves as an intelligent advisor for students who are exploring career paths or programs at Kasetsart University. The system is grounded in official KU curriculum documents provided by faculty, making its guidance specific, accurate, and authoritative rather than approximate.

The system conducts adaptive interest discovery, produces ranked program recommendations with PLO visualizations, provides career path detail for matched

programs, and presents a structured TCAS admission guide specific to each faculty and round.

The system focuses exclusively on students at the pre-admission stage — before any enrollment decision is made.

The core AI architecture combines a RAG pipeline over มคอ.2 curriculum documents with a Neo4j knowledge graph connecting faculties, PLOs, skills, and careers. All recommendations are grounded in retrieved curriculum content, not generated from model weights alone.

1.3.1 Features

1. Interest Discovery: An attribute-based topic grid (8 topics) combined with a single structured follow-up question that builds a personalized student interest profile, grounded in preference elicitation research (Meshi et al., 2023; Bchanged et al., 2018). Used as the basis for all downstream recommendations.
2. Program Recommendation Engine: Ranked list of KU programs matched to the student's interest profile, explained in plain language, covering all KU faculties.
3. PLO Spider Chart Visualizer: Interactive radar chart showing the skill profile a student will develop in any given program, overlaid with the student's interest profile for visual fit assessment.
4. Curriculum Chatbot: RAG-powered Q&A interface over มคอ.2 documents supporting natural language queries in Thai and English about any KU program.
5. TCAS Admission Guide: Structured per-faculty, per-round admission information including GPAX requirements, exam score criteria (TGAT/TPAT/A-Level), portfolio requirements, and deadlines.
6. Saved Profile Dashboard: Optional login to persist interest profile, bookmarked programs, and TCAS deadline tracking across sessions.

1.4 Target Users

KUru serves two primary user groups:

- High school students (pre-admission): Students in Mathayom 4-6 (approximately 15-18 years old) considering KU programs for TCAS applications. This group has no prior KU enrollment and requires guidance from the point of interest discovery through to understanding admission requirements. This is the primary user group for the MVP.
- Students exploring KU programs (pre-admission): Any student at the pre-admission stage who is interested in discovering what KU programs offer, what careers they lead to, and how to apply. This includes high school students preparing for TCAS, gap-year students reconsidering options, and anyone exploring KU programs before making an enrollment decision.

Users may access the system without an account. Login unlocks profile persistence, saved programs, and deadline notifications.

1.5 Terminology

- PLO (Program Learning Outcome): A formally defined competency that graduates of a specific program are expected to have developed, as documented in มคอ.2.
- CLO (Course Learning Outcome): A learning outcome defined at the individual course level, documented in มคอ.3. CLOs contribute to PLOs.
- มคอ.2: The Thai Qualifications Framework program specification document for each accredited university program. Contains PLOs, curriculum structure, and admission requirements.
- มคอ.3: The course-level specification document detailing CLOs, teaching methods, and assessment for each course.
- TCAS: Thai University Central Admission System. The national university admission platform through which students apply to Thai universities in up to four rounds per year.
- TGAT/TPAT/A-Level: Standardized national examinations used as admission criteria in TCAS Rounds 2-4.
- RAG (Retrieval-Augmented Generation): An AI architecture that retrieves relevant passages from a document corpus and provides them as context to a language model before generating a response, grounding output in retrieved evidence.
- Knowledge Graph: A graph database structure in which nodes represent entities (faculties, PLOs, skills, careers) and edges represent relationships between them (a faculty produces certain PLOs, a PLO develops certain skills, a career requires certain skills).
- O*NET: The Occupational Information Network, a US Department of Labor database cataloguing ~900 occupations with structured skill requirements. Used as the career-to-skill data layer.

Chapter 2: Literature Review and Related Work

2.1 Competitor Analysis

Four categories of existing solutions are relevant to this project:

2.1.1 KU Official Information Channels

KU's official website and faculty pages provide static information about programs, curricula, and admission requirements. PLO information is available only within มคอ.2 PDF documents, which are distributed without structured interfaces. TCAS information is scattered across the main KU admissions page, individual faculty pages, and the national mytcas.com platform. No personalization, synthesis, or guidance is offered. The fundamental limitation is that information exists but is not actionable for students making decisions.

2.1.2 General-Purpose LLMs

General-purpose language models such as ChatGPT and Gemini can respond to natural language queries about KU programs, but do so from training data rather than authoritative sources. This results in approximations and hallucinations when asked about specific PLOs, current TCAS requirements, or quota numbers. These models have no persistent memory of the student across sessions and provide no structured outputs such as visualizations or deadline trackers.

2.1.3 University-Specific AI Advisors at Other Thai Institutions

Knowva (KMITL) and ChulaGENIE (Chulalongkorn University) represent the closest competitors to KUru. Both are AI-assisted advising systems built for specific Thai universities. However, neither is available at KU, neither explicitly links PLOs to career skill requirements, and neither provides TCAS-specific admission guidance. ChulaGENIE focuses on course recommendation for enrolled students rather than pre-admission guidance.

2.1.4 Comparison Summary

Feature	KU Website	ChatGPT/Ge mini	Knowva/Chul aGENIE	KUru (ours)
KU-specific curriculum data	Partial (PDF)	None	None	Full (มคอ.2 RAG)
PLO visualization	None	None	None	Spider chart
Career-to-PLO mapping	None	Approximate	Partial	Full graph
TCAS admission guide	Scattered	Approximate	None	Structured
Personalization	None	Session-only	Basic	Progressive profile
Thai language support	Yes	Partial	Partial	Full

Enrolled student skill tracking (Phase 2)	None	None	None	Yes (Phase 2)
---	------	------	------	---------------

2.2 Literature Review

2.2.1 Retrieval-Augmented Generation (RAG)

Lewis et al. (2020) introduced RAG as an architecture combining dense retrieval with sequence-to-sequence generation for open-domain question answering. The key insight is that grounding generation in retrieved evidence substantially reduces hallucination and allows the model to answer questions requiring specific factual knowledge not reliably encoded in model weights. This is directly applicable to KUrU's requirement to answer questions about specific PLO content, admission criteria, and course structures from มคอ.2 documents.

For evaluation, the RAGAS framework (Es et al., 2023) provides automated metrics for RAG pipeline quality including faithfulness, answer relevancy, and context recall, enabling objective assessment of whether the system's answers are grounded in retrieved curriculum content.

2.2.2 Knowledge Graphs for Educational Recommendation

Knowledge graph-based recommendation systems have been demonstrated in educational contexts to enable multi-hop reasoning that purely embedding-based approaches cannot support. Embedding-based retrieval can identify semantically similar items but cannot follow explicit typed relationships such as 'this PLO develops this skill' or 'this career requires this skill level.' The Neo4j graph in KUrU encodes these typed relationships explicitly, enabling recommendation paths that are both explainable and traceable to specific curriculum documents.

2.2.3 Adaptive Quiz and Interest Profiling

KUrU's interest discovery interface is designed around three evidence-based principles from the preference elicitation (PE) and recommender systems literature. First, the system uses explicit attribute-based elicitation rather than item-based elicitation. Research on large-scale onboarding systems demonstrates that item-based approaches—asking users to evaluate individual items directly—are unsuitable when the item corpus is large, because no single item efficiently maps a new user's preference space (Meshi et al., 2023). KUrU instead presents topic clusters (attributes) that map directly to skill vectors, enabling broad preference coverage in minimal interactions. Second, the questionnaire is deliberately minimal. Bchanged et al. (2018) proved at ACM RecSys that an optimized Static Preference Questionnaire (SPQ) can reduce necessary questionnaire length by up to a factor of three compared to standard elicitation methods, while maintaining or improving recommendation quality. KUrU's two-step design—a topic grid followed by a single attribute question—reflects this principle of selecting a minimal yet maximally diverse question set. Third, the system addresses the new user cold-start problem (Bobadilla et al., 2014) through this explicit elicitation phase, then transitions to progressive implicit profiling. Behavioral signals including program page visits, chatbot queries, and saved items continuously refine the user's interest vector without additional questioning (Kanoje et al., 2015; Schilder et al., 2023). The structured grid interface is supported by de Vries et al. (2024), who found in a controlled user study that

high-guidance elicitation—presenting structured options rather than open-ended prompts—produces significantly higher recommendation match scores than low-guidance alternatives. This work applies established cold-start PE methods from content recommendation domains (music, film) to academic pathway advising, which constitutes a domain contribution: the user population (Thai high school students), the item space (university programs defined by PLOs), and the stakes (multi-year academic commitment) differ substantially from prior application contexts.

Chapter 3: Requirement Analysis

3.1 Stakeholder Analysis

- High school students (prospective KU applicants): Primary users. Need personalized program recommendations, PLO-based skill previews, career path information, and structured TCAS guidance. Low technical proficiency expected; interface must be accessible and engaging.
-
- KU Faculty / Academic Advisors: Data providers. Provide มคอ.2 and มคอ.3 documents. Indirect beneficiaries through reduced advising load and better-prepared incoming students.
- KU Admissions Office: Indirect stakeholders. Benefit from better-informed applicants and reduced mismatched enrollments.
- Development Team: Two SKE students responsible for design, development, and evaluation within one semester.

3.2 User Stories and Use Cases

3.2.1 High School Student User Stories

- As a high school student, I want to discover which KU programs match my interests so that I can make an informed TCAS application.
- As a high school student, I want to see what skills I will graduate with from a specific program so that I can assess whether it aligns with my career goals.
- As a high school student, I want to understand the TCAS admission requirements for a program I am interested in so that I know what I need to prepare.
- As a high school student, I want to ask questions about a program's curriculum in Thai so that I can get specific answers without reading long PDF documents.
- As a high school student, I want to save programs I am considering so that I can return to compare them later.

3.2.2 Additional User Stories

- As a returning visitor, I want my saved interest profile and bookmarked programs to be available when I come back so that I do not have to start over.
- As a parent, I want to browse KU programs and understand what careers they lead to so that I can support my child's decision.

3.3 Use Case Diagrams

[Figure placeholder: UML Use Case Diagram showing Guest and Registered User actors with use cases: Discover Interests, View Program Recommendations, Explore PLO Chart, Query Curriculum Chatbot, View TCAS Guide, Save Profile (registered only), Track TCAS Deadlines (registered only)]

3.4 User Interface Design

The interface follows a mobile-first web design using Next.js, Tailwind CSS, and Shadcn/UI. Key screens include:

- Landing page: Value proposition with two entry paths (interest discovery for undecided students; career search for students with a goal in mind). Bilingual Thai/English throughout.
- Interest discovery: Attribute-based topic grid (8 topics) for multi-select, followed by one structured follow-up question. Progress indicator. Results in an animated interest profile summary before recommendations are shown.
- Program explorer: Ranked faculty cards with fit score explanation. Tap to expand into PLO spider chart, course overview, and career path detail.
- TCAS guide: Faculty-specific breakdown of applicable rounds, score requirements, portfolio criteria, and deadlines.
- Curriculum chatbot: Conversational interface supporting Thai and English queries, with source citations from ๓๑๑.2 documents.
- Saved profile dashboard: Bookmarked programs, TCAS deadline tracker, interest profile summary. Requires login.

Chapter 4: Software Architecture Design

4.1 System Architecture Overview

KUru is a three-tier web application with a distinct AI layer. The frontend (Next.js) communicates with a Python FastAPI backend, which orchestrates two AI subsystems: a RAG pipeline for curriculum Q&A and a graph-based recommendation engine. Both subsystems read from a shared data layer consisting of Supabase (PostgreSQL + pgvector for embeddings) and Neo4j (knowledge graph on Railway).

[Figure placeholder: System architecture diagram showing: Next.js frontend → FastAPI backend → RAG Engine (Supabase pgvector + Gemini) and Recommendation Engine (Neo4j + interest profile) → Data Ingestion Pipeline → External sources (มคอ.2 PDFs, O*NET, TCAS data). Also showing: Supabase Auth for user profiles and Gemini API for LLM calls.]

4.2 Software Design

4.2.1 RAG Pipeline Sequence

When a user submits a query to the curriculum chatbot: (1) the query is embedded using Gemini text-embedding-004; (2) the embedding is used to perform a cosine similarity search against the pgvector store of มคอ.2 document chunks; (3) the top-k retrieved chunks are assembled into a prompt context along with the user query and the conversation history; (4) Gemini 2.0 Flash generates a grounded response in the requested language; (5) source citations are appended to the response.

4.2.2 Recommendation Pipeline Sequence

When a student completes the interest discovery flow: (1) topic grid selections and follow-up answer are mapped to a weighted skill cluster vector; (2) the vector is embedded and used for pgvector similarity search to retrieve candidate programs; (3) for each candidate, Neo4j is queried to retrieve its PLO-to-skill subgraph; (4) a fit score is computed as the cosine similarity between the student's skill vector and the program's PLO skill profile; (5) the top-N ranked programs are returned with plain-language explanations generated by Gemini.

4.3 AI and Data Design

4.3.1 Data Sources and Collection

- KU curriculum documents (มคอ.2): Provided directly by KU faculty advisors for all KU programs. Thai-language PDFs of varying length (typically 20-80 pages per program). These are the primary knowledge base for both the RAG pipeline and the PLO extraction pipeline. Documents are version-tagged by academic year and re-ingested when updated.
- O*NET occupational database: Publicly available from the US Department of Labor at onetonline.org. Contains structured skill requirements for approximately 900 occupations, used to populate the career-to-skill layer of the knowledge graph. Downloaded as structured data (CSV/JSON) and mapped to the Thai career taxonomy used in the interest discovery system.

- TCAS admission data: Collected from mytcas.com and individual KU faculty admission pages. Self-collected and structured into a per-faculty, per-round schema. Updated manually at the start of each TCAS cycle and version-tagged by year.

Data preprocessing for มคอ.2 documents involves: (1) text extraction from PDF using PyMuPDF; (2) section-aware chunking (PLO sections, course descriptions, and learning outcomes are chunked separately to preserve semantic coherence); (3) embedding generation using Gemini text-embedding-004; (4) storage in Supabase pgvector with metadata tags for faculty, program, document version, and section type.

4.3.2 Model Design

The primary language model is Gemini 2.0 Flash, selected for its strong Thai language performance, native PDF document understanding, 1M token context window, and cost efficiency relative to GPT-4o. Direct Gemini API calls are used without LangChain or similar orchestration frameworks, to reduce latency and maintain explicit control over prompt structure.

Embeddings use Gemini text-embedding-004, a multilingual model that produces consistent semantic representations for both Thai and English text, enabling cross-language retrieval where a Thai query can retrieve an English document chunk and vice versa.

The Neo4j knowledge graph uses a schema with four node types: Faculty, PLO, SkillCluster, and Career. Edges represent: Faculty `-[HAS_PLO]->` PLO, PLO `-[DEVELOPS]->` SkillCluster, Career `-[REQUIRES]->` SkillCluster. This schema supports the multi-hop queries required for recommendation: given a student's SkillCluster profile, find Faculties whose PLOs develop those clusters. The interest discovery interface employs explicit attribute-based preference elicitation (PE) rather than item-based elicitation. This choice is grounded in research demonstrating that item-based approaches are unsuitable for onboarding when the item corpus is large (Meshi et al., 2023). The questionnaire is deliberately minimal: Bchanged et al. (2018) at ACM RecSys demonstrated that an optimized preference questionnaire can reduce necessary question length by up to a factor of three while maintaining or improving recommendation quality. The two-step design—a topic grid followed by a single attribute question—reflects this principle. The system addresses the new user cold-start problem (Bobadilla et al., 2014) through this explicit elicitation phase, then transitions to progressive implicit profiling: behavioral signals such as program page visits, chatbot queries, and saved items continuously refine the user's interest vector without additional questioning (Kanoje et al., 2015; Schilder et al., 2023). The choice of a structured grid interface over open-ended input is supported by de Vries et al. (2024), who found that high-guidance elicitation produces significantly higher recommendation accuracy than low-guidance alternatives.

The baseline for comparison is BM25 keyword retrieval over the same มคอ.2 corpus without embedding or LLM generation. This establishes a performance floor against which RAG improvement is measured.

4.3.3 Evaluation

- RAG pipeline quality: RAGAS framework metrics — Faithfulness (proportion of claims in the answer supported by retrieved context), Answer Relevancy (semantic similarity between answer and question), Context Recall (proportion of relevant information retrieved). Target: Faithfulness > 0.80, Answer Relevancy > 0.75.
- Recommendation quality: Mean Reciprocal Rank (MRR) and NDCG@5 evaluated on a manually curated test set of 30-50 student interest profiles with ground-truth faculty matches validated by faculty advisors or senior students. Target: MRR > 0.60.
- User experience: Task completion rate and System Usability Scale (SUS) score from user testing with 10-15 high school students or KU first-year students. Target: SUS score > 70 (acceptable).

Chapter 5: Software Development

5.1 Software Development Methodology

The project uses an Agile-inspired iterative approach adapted for a two-person team over one semester. Development is organized into two-week sprints aligned with the phases in the project schedule. Each sprint ends with a working, demonstrable increment. A shared Kanban board (Notion) tracks tasks, and GitHub is used for version control with a feature-branch workflow. Weekly check-ins with the project advisor provide feedback and course correction.

Given the data dependency at the start of the project (1000+ documents from faculty), the first two weeks are allocated to data collection and pipeline setup, with feature development beginning only once the core data layer is operational.

5.2 Technology Stack

- Frontend: Next.js 14 (App Router), Tailwind CSS, Shadcn/UI. Chosen for familiarity, strong TypeScript support, and built-in i18n routing for Thai/English.
- Backend: Python FastAPI. Chosen for Python ecosystem access (PyMuPDF for PDF processing, sentence-transformers, neo4j driver) and async performance.
- Vector database: Supabase (PostgreSQL + pgvector extension). Stores document embeddings, user profiles, and TCAS structured data. Supabase Auth handles authentication.
- Graph database: Neo4j on Railway. Stores the PLO-Skill-Career knowledge graph. Queried via Cypher from the FastAPI backend.
- AI: Gemini 2.0 Flash (generation), Gemini text-embedding-004 (embeddings), accessed via Google AI SDK for Python.
- Data processing: PyMuPDF (PDF extraction), custom Python chunking pipeline.
- Deployment: Vercel (frontend), Railway (FastAPI backend + Neo4j).

5.3 Coding Standards

- Python: PEP 8, type hints on all function signatures, docstrings for all public functions.
- TypeScript: ESLint with Next.js recommended config, Prettier for formatting.
- Git: Conventional commits format (feat/fix/chore/docs). Feature branches merged via pull request with at least one review.
- Environment: All secrets managed via .env files, never committed. Supabase and Gemini API keys stored as environment variables.

5.4 Schedule

Period	Phase	Deliverables
--------	-------	--------------

Weeks 1–2	Data collection	Finalize มคอ.2 docs from professors, scrape TCAS data, download O*NET dataset
Weeks 3–4	Data pipeline	PDF extraction, chunking, embedding into Supabase pgvector; Neo4j schema + pilot data
Weeks 5–6	Core AI	RAG pipeline (retrieval + Gemini generation); recommendation engine (graph + interest profile)
Weeks 7–9	Frontend	Interest discovery UI, PLO spider chart, TCAS guide, chatbot interface, saved profile
Weeks 10–11	Evaluation	RAGAS evaluation, MRR/NDCG test set, user testing with 10–15 high school students
Week 12	Demo & docs	Final demo preparation, documentation, and project report

5.5 Progress Tracking

[GitHub contribution graph and sprint burndown charts to be included here as the project progresses.]

Chapter 6: Deliverables

6.1 Software Solution

[GitHub repository link to be added. Screenshots of each application screen to be included upon completion.]

The delivered software will include: a deployed web application accessible at a public URL; the data ingestion pipeline for มคอ.2 PDF processing; the Neo4j knowledge graph populated with all KU programs; the RAG pipeline and recommendation engine; and the six core MVP features described in Chapter 1.

6.2 Test Report

Testing will cover three levels:

- Unit tests: Core AI functions (chunking, embedding, retrieval, graph query) tested with pytest. Coverage target: 70% of backend functions.
- Integration tests: End-to-end API tests for each major user flow (interest discovery to recommendation, chatbot query to grounded response). Run automatically on each pull request via GitHub Actions.
- User tests: Manual usability testing sessions with 10-15 participants, using a structured task script and SUS questionnaire. Results tabulated and reported in the test report.

Chapter 7: Conclusion and Discussion

7.1 Summary of Achievements

[To be completed at project conclusion. Will summarize delivered features, evaluation results, and degree to which the problem statement was addressed.]

7.2 Known Limitations

- TCAS data currency: TCAS admission requirements change each academic year. The system requires manual data updates at the start of each TCAS cycle and does not automatically detect changes.
- CLO-level mapping: The system uses program-level PLO data (มคอ.2) only. Course-level CLO data (มคอ.3) is not ingested in this project. Future extensions requiring CLO mapping would need a separate data pipeline for มคอ.3 documents.
- O*NET Thai career mapping: O*NET occupational data is in English and uses US career taxonomy. Manual mapping to Thai career titles and KU graduate employment patterns is required and introduces approximation.
- Portfolio scorer: Designed and documented but not implemented in the MVP due to data dependency on structured per-faculty portfolio rubrics from the admissions office.
- Evaluation ground truth: The MRR evaluation test set is manually curated with limited size (30-50 profiles), which constrains the statistical reliability of the recommendation quality metrics.

7.3 Future Work

- Enrolled student features (Phase 2): Extend the system to serve currently enrolled KU students with skill progress tracking, elective recommendations, and career readiness reporting. This would require มคอ.3 CLO ingestion and a course enrollment interface. The current knowledge graph schema supports this extension without architectural changes.
- Portfolio scorer: Build the AI-assisted portfolio evaluation feature once structured admission rubrics are available from the KU admissions office.
- Automated TCAS data refresh: Implement web scraping with change detection to identify when TCAS requirements are updated, triggering a manual review and re-ingestion cycle.
- Thai career taxonomy alignment: Develop a systematic mapping between O*NET occupations and THACO (Thai Standard Occupational Classification) to improve career recommendation accuracy for Thai students.
- Longitudinal evaluation: Track recommendation quality by following up with admitted students after one year to assess whether their actual program experience matched the system's prediction.

References

Es, S., James, J., Espinosa-Anke, L., & Schockaert, S. (2023). RAGAS: Automated evaluation of retrieval augmented generation. arXiv preprint arXiv:2309.15217.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 9459-9474.

Ministry of Education, Thailand. (2552). Thai Qualifications Framework for Higher Education (TQF: HEd). Office of the Higher Education Commission.

My TCAS. (2567). TCAS68 Admission Guidelines. Retrieved from <https://www.mytcas.com>

National Center for O*NET Development. (2024). O*NET OnLine. Retrieved from <https://www.onetonline.org>

Bobadilla, J., Ortega, F., Hernando, A., & Gutierrez, A. (2014). Dealing with the new user cold-start problem in recommender systems: A comparative review. *Information Systems*, 58, 68–83.

<https://doi.org/10.1016/j.is.2014.09.005>Bchanged, A., Krichene, W., & Rendle, S. (2018). Preference elicitation as an optimization problem. In *Proceedings of the 12th ACM Conference on Recommender Systems (RecSys '18)* (pp. 172–180). ACM.

<https://doi.org/10.1145/3240323.3240352>de Vries, R., Banovic, N., & Munteanu, C.

(2024). The effect of preference elicitation methods on the user experience in conversational recommender systems. *Computer Speech & Language*, 101704.

<https://doi.org/10.1016/j.csl.2024.101704>Kanoje, S., Girase, S., & Mukhopadhyay, D.

(2015). User profiling trends, techniques and applications. *International Journal of Advance Foundation and Research in Computer*, 1(12). (Surveyed in: *User Modeling and User Profiling: A Comprehensive Survey*, arXiv:2402.09660, 2024.)

Meshi, O., Feldman, J., Yang, L., Scheetz, B., Cai, Y., Bateni, M., Salisbury, C., Aggarwal, V., & Boutilier, C. (2023). Preference elicitation for music recommendations. In *ICML 2023 Workshop: The Many Facets of Preference-Based Learning*.

Schilder, E., Prasse, P., & Scheffer, T. (2023). A global user profile framework for effective recommender systems. *Multimedia Tools and Applications*, 83, 42619–42643.

<https://doi.org/10.1007/s11042-023-17436-w>